# 计算机体系结构

## 05. 缓存性能的高级优化

李建华
计算机与信息学院

**The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburg, and eth. The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.**

# 优化缓存性能的高级方法

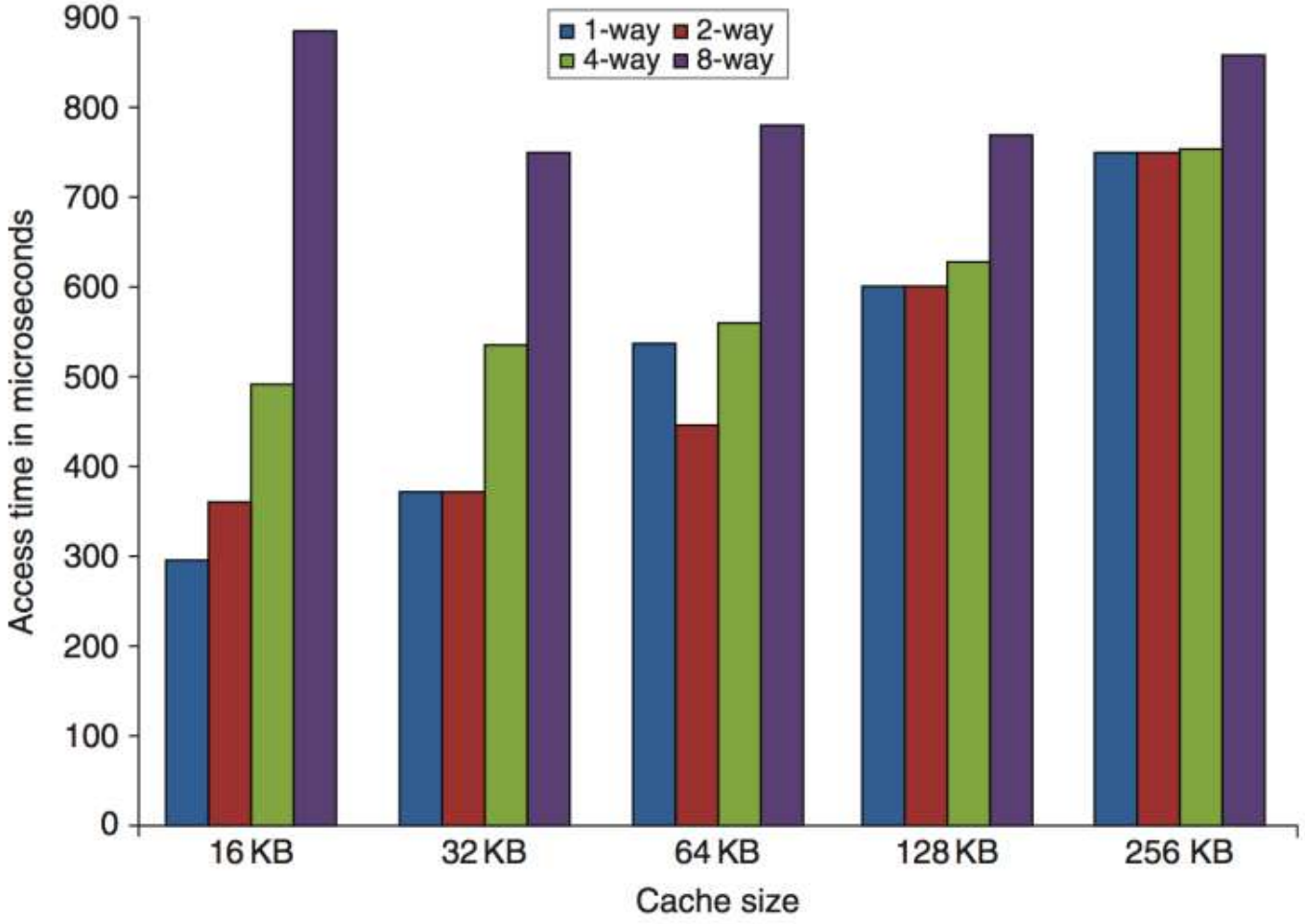
- **缩短命中时间**：采用小且简单的一级缓存、路预测
  - 这两种技术通常还能**降低功耗**
- **增加缓存带宽**：缓存访问流水化、多Bank缓存、非阻塞缓存
- **降低缺失开销**：关键字优先、写缓冲合并
- **降低缺失率**：基于编译器的优化技术
- **通过并行性降低缺失开销或缺失率**：硬件预取、编译器预取
  - 这些技术通常会增加功耗，因为预取的数据不一定有用。

# 1. 小而简单的L1缓存

- Smaller is faster，缓存越小，存取速度越快
  - L1高速缓存的大小近些年稍微增加或根本没有增加
    - 例如，Alpha21164具有 8KB指令和 8KB数据缓存+ 96KB二级缓存
    - 例如，3代AMD微处理器的L1大小相同：K6，Athlon，和Opteron

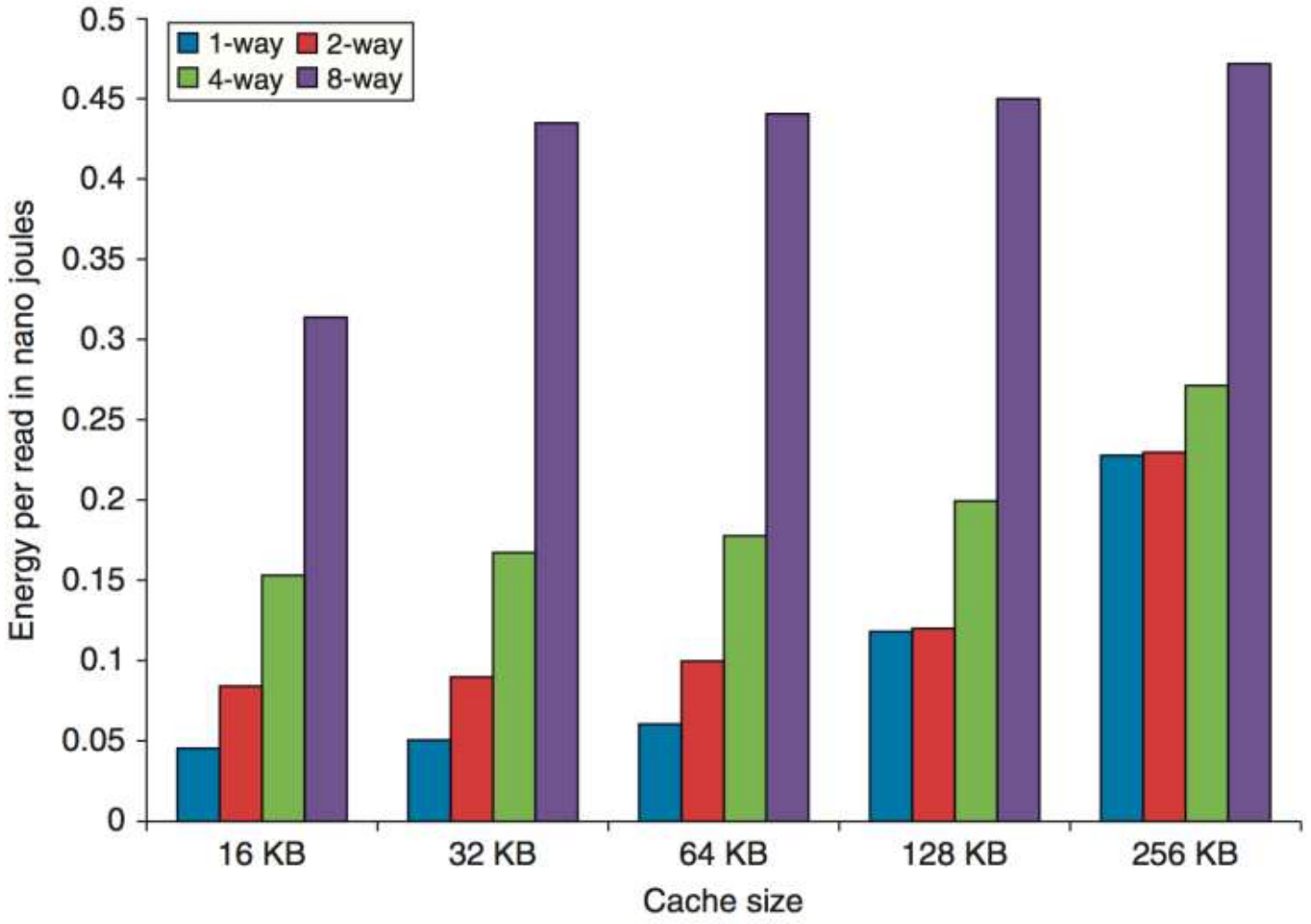
- 为什么简单的L1$更高效?
  - 直接映射缓存可以将标记检查与数据传输重叠，从而有效地减少命中时间;
  - 较小的相联度通常会降低功耗，因为需要访问的缓存行较少。

# Case size -> AMAT




通常，随着缓存大小和相联度的增加，访问时间会延长。数据来源于cacti-6.5!

# Cache size -> Energy




通常，随着缓存大小和相联度的增加，读功耗会增加。数据来源于cacti-6.5！

# 2. Way Prediction

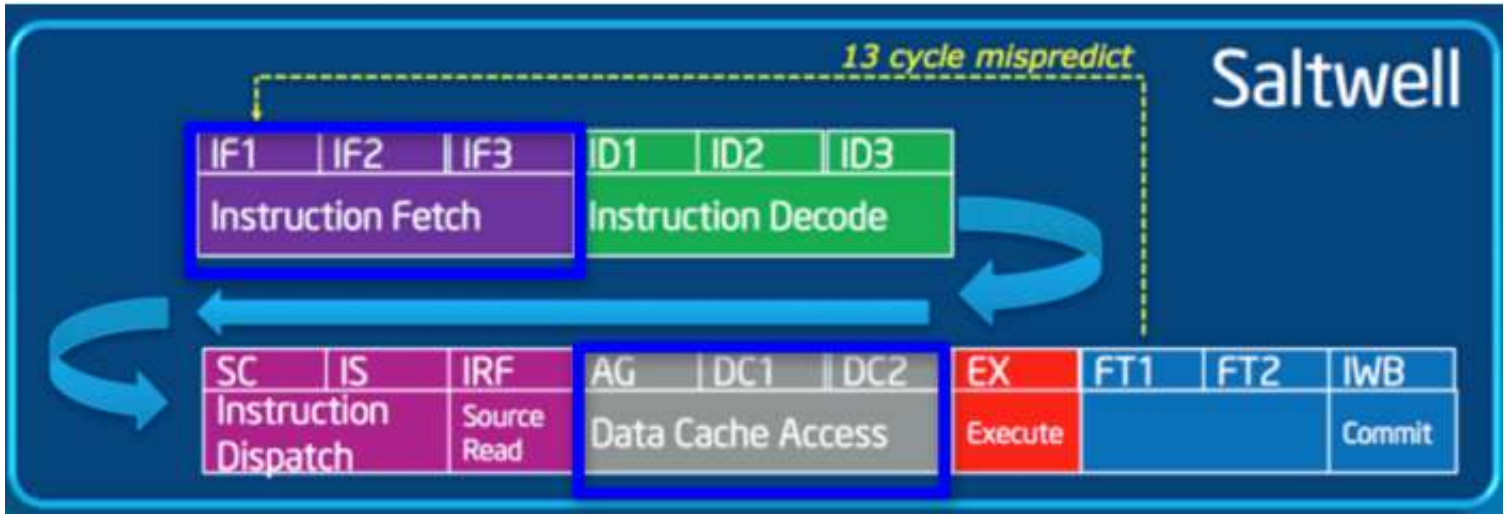
- 如何既能拥有直接映射的快速命中，又能具备二路组相联缓存的更低缺失率?
- 路预测：在缓存中保留额外的位来预测下次访问的“路”(组内的块)
  - 这样，多路选择器提前设置以选择所需的块；在该周期内仅进行一次标签比较（与读取数据并行）。
  - 如果预测的不对，那么在下一个周期检查其他块是否有匹配。
  - 预测的正确率约85%

**也能应用到高相联度的缓存架构!**



# 3. 用流水线提升访问缓存带宽 (1)

- 将缓存的访问流水线化
  - 那么L1缓存的命中就需要多个时钟周期



# 3. 用流水线提升访问缓存带宽 (2)

- 优点：更快的时钟周期，命中时间变长
- 示例：Intel处理器I-cache访问时间
  - Pentium：1个时钟周期，1990年中期
  - Pentium Pro~ Pentium III：2个时钟周期，1990年中期至2000年
  - Pentium4：4个时钟周期，2000年代
  - Intel Core i7：4个时钟周期
- 缺点：增加了流水线的阶段，会导致：
  - 分支预测错误时的penalty增加
  - 读指令发出到获取数据之间的延迟会增加

注意：流水线只能提升缓存访问带宽，不会缩短单次访问缓存所需的时间。

# 4. 用非阻塞缓存提升访存带宽 (1)

- **问题分析：**如果处理器可以发出多个缓存访问请求，那么若前一个缺失还未处理完时，后续的访问能否继续处理？ **好处很明显：较少的停顿。**
- **目标 1：**当有一个未处理完的缓存缺失，允许继续访问缓存，而不是暂停。
- **目标 2：**使得多个缺失可以并行处理，提升存储级并行性 (MLP)。
- **解决方法：** Non-blocking or lockup-free caches (非阻塞缓存技术)
  - David Kroft, "Lockup-Free Instruction Fetch/Prefetch Cache Organization," ISCA 1981.
  - ISCA 25周年经典论文之一

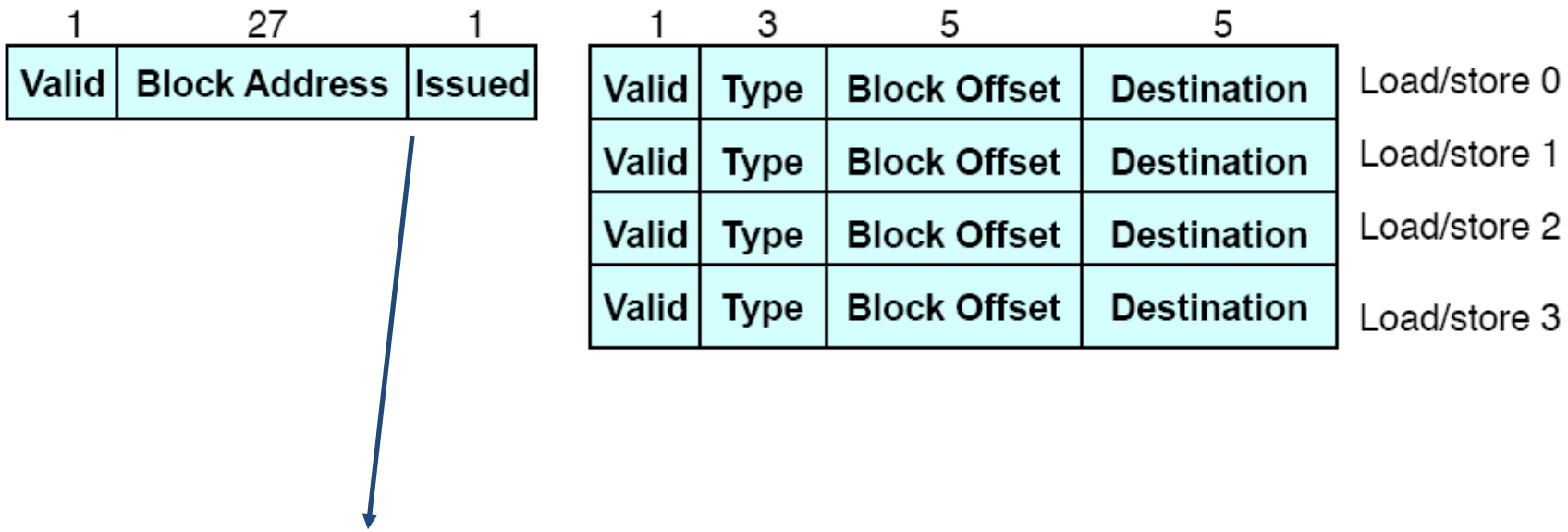
# 4. 用非阻塞缓存提升访存带宽 (2)

- **想法：**用MSHR来记录正在处理的缺失请求的相关信息 (状态和数据)，使得后续的访问请求尽量不用暂停。
  - MSHR的全称是：Miss Status Handling/Holding Register (MSHR)
  - 那么：访问缓存发生缺失时，需要检查MSHR以确认是否已经有访问相同的block的缺失请求正在处理。
    - 如果MSHR中有，那么就没有必要产生一个新的缺失请求；
    - 如果MSHR中有并且所需的数据已返回 (在MSHR里面), 那么数据直接发给请求者即可；
    - 如果有，而所需的数据未返回，则尝试登记当前访问者的信息；
    - 如果MSHR中没有，则**尝试产生**一个新的缺失请求，在MSHR里记录相关信息。[**此时，可能MSHR用完了，因资源冲突而阻塞**]

# MSHR解析

- MSHR是**一组寄存器**，用于记录：
  - 正在处理的缓存访问缺失请求
  - 正在处于等待状态的读写请求，访问的是一个已经在缺失状态的block。
- MSHR条目中记录的信息：
  - valid bit
  - block address (用于后续的请求进行匹配)
  - control/status bits
    - Prefetch? issued to memory? which sub-blocks have arrived? etc…
  - data for each sub-block
  - 对每个处于等待状态的读写请求，记录：
    - valid, type, byte offset in block, destination register or store buffer entry address

# MSHR 数据结构




The data for each sub-block is not shown in the figure

# MSHR 操作流程

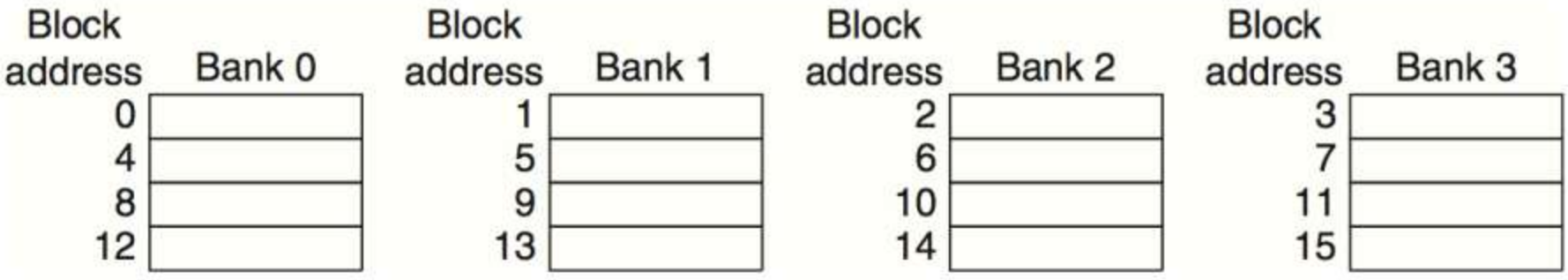
- 发生缓存缺失时：
  - 查找MSHR看是否有其它处于等待状态的请求访问相同的block?
    - 若找到：尝试在同一个MSHR条目里分配一个load/store条目；若没有空闲的load/store条目，则暂停 — **结构冒险**；
    - 若没找到：尝试分配一个新的MSHR条目；若没有空闲的MSHR，则暂停 – **结构冒险**；
- 当一个sub-block从下一级存储器返回：
  - 确定哪些loads/stores在等待该 sub-block
    - 将sub-block传给相应的load/store unit
    - 释放MSHR条目中相应的load/store条目
  - 将sub-block写到缓存**或者**MSHR中的data字段中
  - 若返回的是最后一个sub-block，在将子块写到缓存中，然后释放相应的MSHR条目。

# 5. 多bank架构提升缓存访问带宽

- 与其将缓存设计为单个整体块，不如将其划分为多个独立的存储体 (Bank)，以支持并行访问。
  - Arm Cortex-A8的二级缓存支持1到4个bank；
  - Intel Core i7的一级缓存有四个bank（以支持每个时钟周期最多 2 次缓存访问），二级缓存有八个bank。

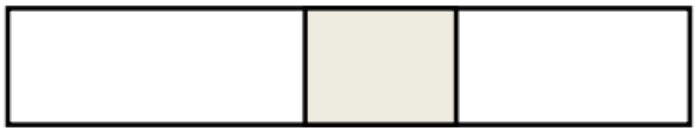
# 顺序交叉映射

- 当访问对象连续地分布在各个bank上时，效果最佳。
  - 地址到存储体的映射 会 影响内存系统的行为
- 一种高效且简单的映射方法：顺序交叉
  - 将块地址按顺序分布在各个bank上
  - 例如，bank *i* 包含所有地址对 *n* 取模等于 *i* 的块



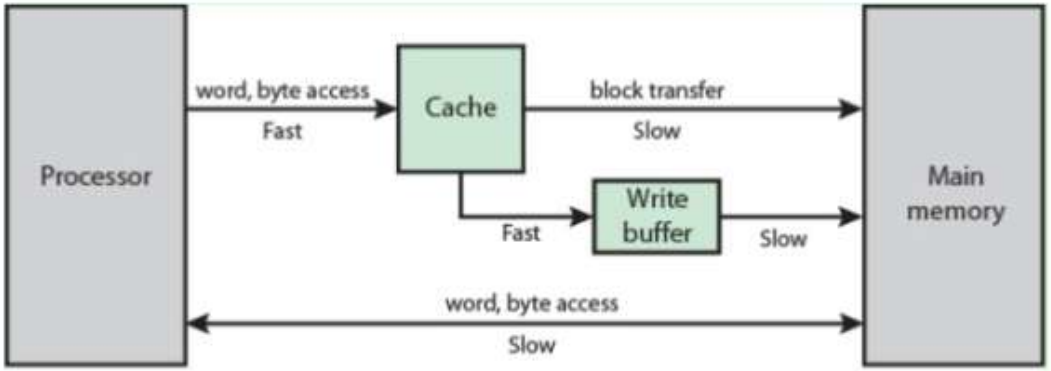
# 6. Early start & Critical word first

- 缓存访问缺失时，不要等整个缓存块都准备好再重启 CPU。
- **关键字优先：**优先从内存请求缺失的字，立即将其发送到 CPU；在填充块的其余部分时，让 CPU 继续运行。
- **提前重启：**一旦缓存块中被请求的字到达，就立即发送到 CPU ，让其继续执行，不用等待。
- 这2个技术更适用于大的block size设置，在局部性较好的访存模式下，非常高效。



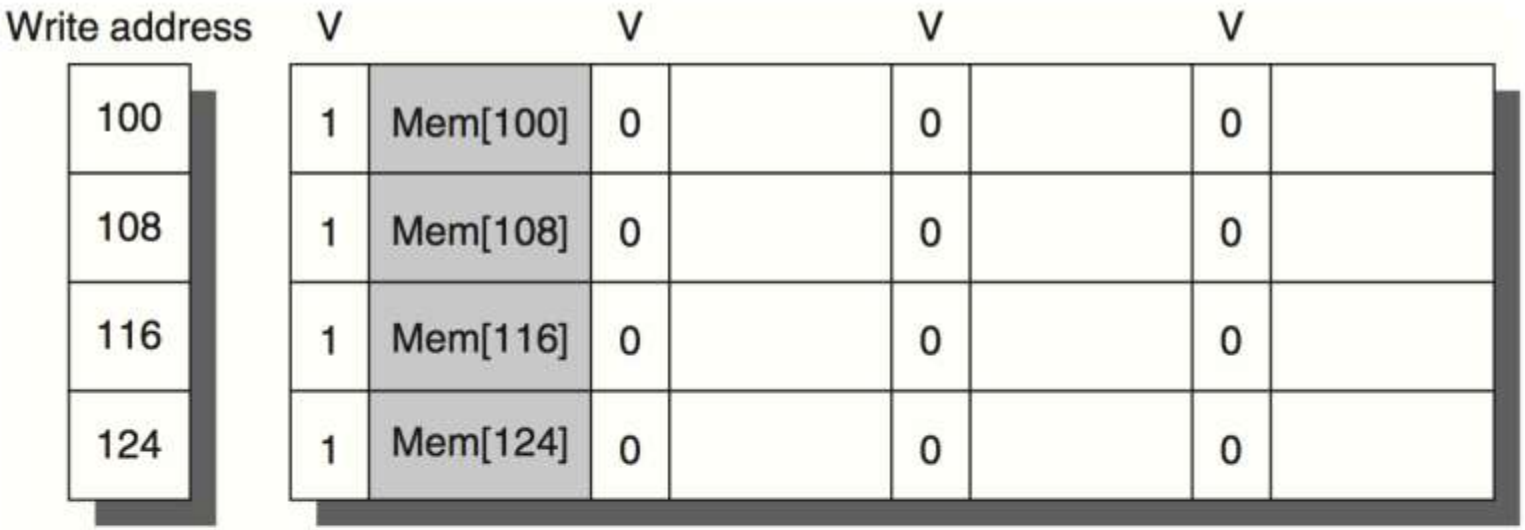
# 7. 使用写缓冲**聚合**写操作

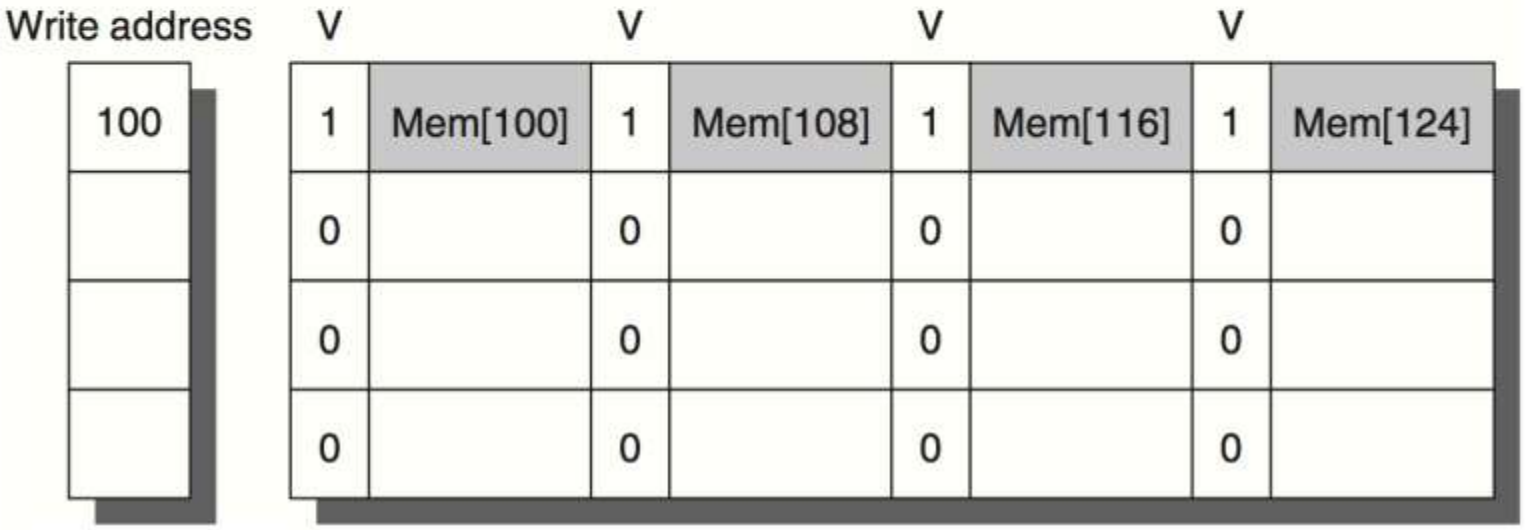
- 写缓冲 (WB) 可以使CPU不用等待写操作完成，也能继续执行任务。
- WB合并写操作：
  - 若缓冲中包含已修改的块，可通过检查地址来判断新数据是否与某一写缓冲条目（的数据）匹配。
  - 若匹配，则将新数据与该条目（的数据）合并。




- 对于**写直达缓存**中的顺序写操作，增大缓存块大小会更高效。

# 写缓冲示例


Write address
V
V
V
V
100
108
116
124
1
Mem[100]
0
1
Mem[108]
0
1
Mem[116]
0
1
Mem[124]
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0

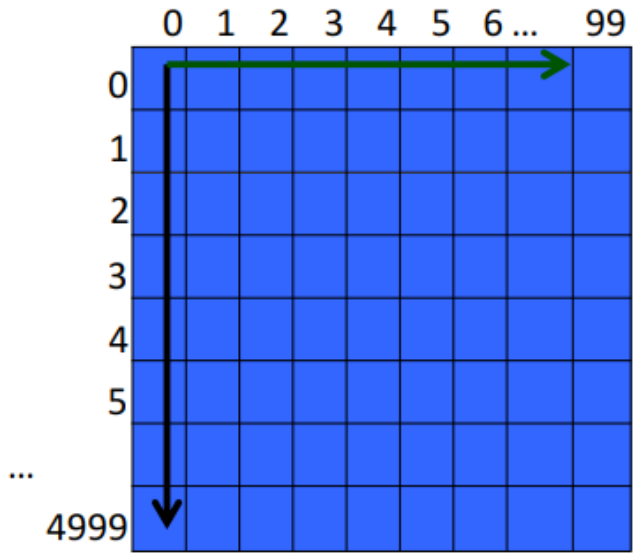

Write address
V
V
V
V
100
1
Mem[100]
1
Mem[108]
1
Mem[116]
1
Mem[124]
0
0
0
0
0
0
0
0
0
0
0
0

# 8. 基于编译器的优化方法

- 重构数据访问特征
- 重构数据在存储中的布局
  - Loop interchange
  - Data structure separation/merging
  - Loop fusion
  - Loop Blocking
  - ……

# ① Loop Interchange

- 示例: 如果是**列式存储**
  - x[i+1,j] 在内存中位于 x[i,j] 之后
  - x[i,j+1] 在内存中距离 x[i,j] 较远




Poor code

```
for i = 1, rows
    for j = 1, columns
        sum = sum + x[i,j]
```

Better code

```
for j = 1, columns
    for i = 1, rows
        sum = sum + x[i,j]
```

- 这个方法叫：loop interchange

Demo

# ② Array merging/splitting (1)

```
struct Node {
    struct Node* next;
    int key;
    char [256] name;
    char [256] school;
}

while (node) {
    if (node→key == input-key) {
        // access other fields of node
    }
    node = node→next;
}
```

- 基于指针的数据访问
  - 例如，链表的访问
- 假定一个具有主键的巨大链表 (1M 个节点)
- Q：为什么左侧的代码执行时缓存的命中率较低?
  - 因为“其它域” 占据了缓存行中大部分空间；然而，它们很少被访问。

如何优化性能?

# ② Array merging/splitting (2)

```
struct Node {
    struct Node* next;
    int key;
    struct Node-data* node-data;
}

struct Node-data {
    char [256] name;
    char [256] school;
}

while (node) {
    if (node→key == input-key) {
        // access node→node-data
    }
    node = node→next;
}
```



- **优化方法：**将频繁访问的数据剥离开，单独组成数据结构；
- 谁可以开展此类优化?
  - 程序员 —> 最方便
  - 编译器 —> 可以提供帮助
    - Profiling
  - 硬件? 可行，但比较困难。

# ③ Loop fusion

```
/* Before */
for (i = 0; i < N; i = i+1)
   for (j = 0; j < N; j = j+1)
    a[i][j] = 1/b[i][j] * c[i][j];

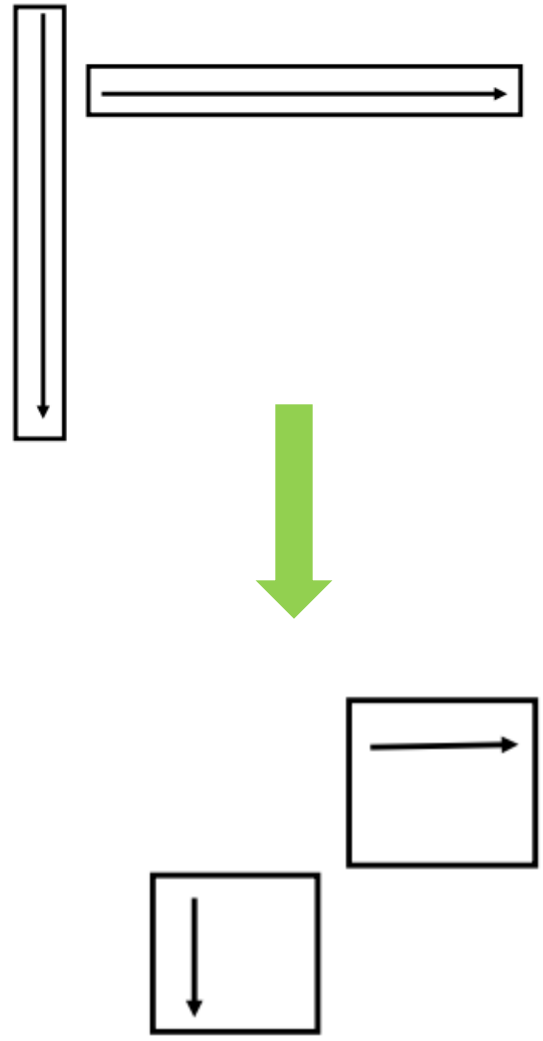
for (i = 0; i < N; i = i+1)
   for (j = 0; j < N; j = j+1)
    d[i][j] = a[i][j] + c[i][j];

/* After */
for (i = 0; i < N; i = i+1)
   for (j = 0; j < N; j = j+1)
   {a[i][j] = 1/b[i][j] * c[i][j];
    d[i][j] = a[i][j] + c[i][j];}
```

- 改善了访问a与c的空间局部性：
  - Before：每次访问2 次缺失，After：每次访问 1 次缺失

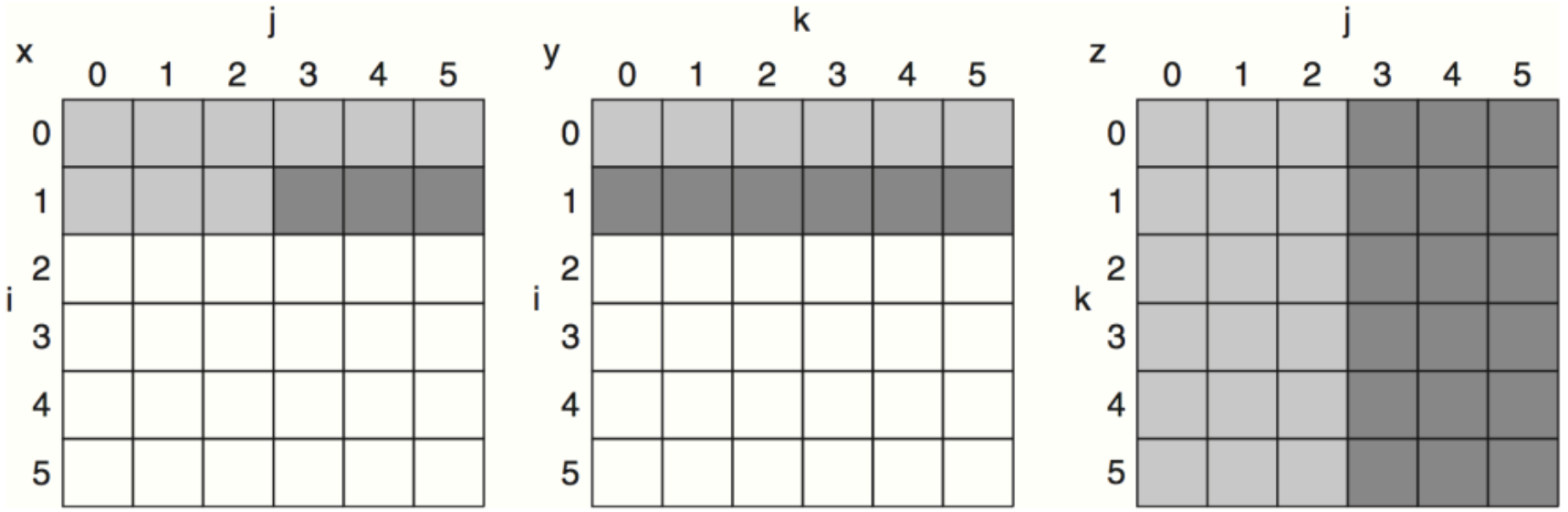
# ④ Loop blocking

```
/* Before */
for (i = 0; i < N; i = i+1)
   for (j = 0; j < N; j = j+1)
      {r = 0;
       for (k = 0; k < N; k = k+1){
         r = r + y[i][k]*z[k][j];};
       x[i][j] = r;
      };
```



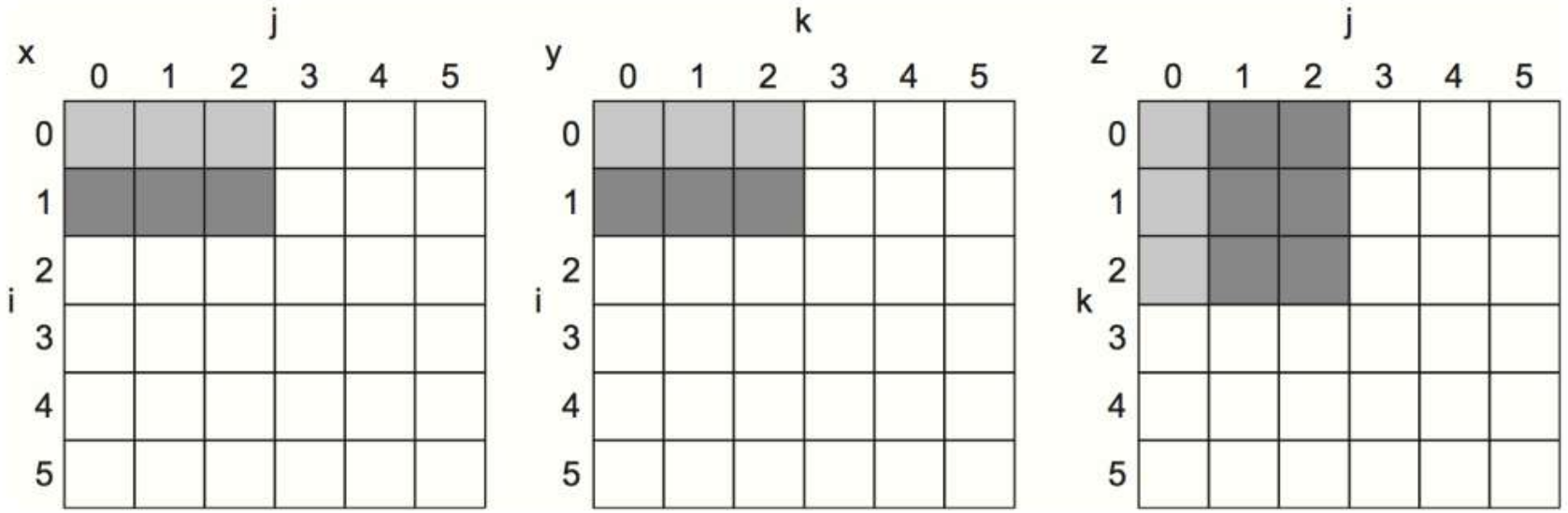
- 两个内层循环：
  - 读取z[] 的所有N×N个元素
  - 反复读取y[]中1行的N个元素
  - 写入x[]中1行的N个元素
- 容量缺失是N和缓存大小的函数：
  - 容量至少为$2N^3+N^2$，才不会有缺失。

# 未使用blocking时数组访问的Snapshot




- 当N=6，i=1时，三个数组的访问过程：
  - 白色块，未被访问过；
  - 浅灰色块，之前被访问过；
  - 深灰色块，最近被访问过。
- 需要重复读取y、z中的块，来计算出x中的块。

# 使用blocking时访问数组的snapshot




- 当N=6，i=1，B=3时的数组访问过程：
  - 与上一页的访问过程相比，这种方式在一段时间内访问的数据量下降了。
  - 一个较小的缓存，就能容纳这种场景所访问的数据。

# 采用blocking策略的code

```
/* After */
for (jj = 0; jj < N; jj = jj+B)
   for (kk = 0; kk < N; kk = kk+B)
      for (i = 0; i < N; i = i+1)
          for (j = jj; j < min(jj+B-1,N); j = j+1)
           {r = 0;
            for (k = kk; k < min(kk+B-1,N); k = k+1) {
               r = r + y[i][k]*z[k][j];};
            x[i][j] = x[i][j] + r;
           };
```

- B称为block factor
- 你可以做实验来分析loop blocking的效果
  - **实验一的自选内容**

# 9. 基于硬件的prefetching

- 在处理器请求数据之前，由硬件进行预取。
  - 指令和数据都可以被预取
  - 预取的数据要么直接写入缓存，要么写入一个访问速度快的专门的缓冲区（prefetch buffer）。
- 指令预取：
  - 通常，CPU 在缺失时会获取 **2 个块**：当前被请求的块和下一个块。
  - 被请求的块进入指令缓存，预取的块进入预取缓冲区。
- 数据预取：
  - Pentium 4 可以从多达 8 个不同的 4KB 页面的 8 个流中预取数据到 L2 缓存。
    - 如果对一个页面连续发生 2 次 L2 缓存缺失，且这些缓存块之间的距离小于 256 字节，就会触发预取。
  - Intel Core i7 支持向 L1 和 L2 进行硬件预取，最常见的预取情况是访问下一行触发预取。

# 关于硬件预取

- 其意图是充分利用原本可能会被闲置的内存带宽
  - 如果预取影响了按需缺失（指处理器实际请求数据时发生的缓存缺失）的处理，反而可能降低性能。
  - 此外，当预取的数据未被使用，或 预取的数据在使用之前就被替换时，预取就是无用的。
    - 同时，无用的预取浪费了带宽和功耗。

**A Primer on Hardware Prefetching**

Babak Falsafi
EPFL
Thomas F. Wenisch
University of Michigan

# 10. 编译器控制的预取

- 编译器插入预取指令，以便在处理器**真正需要数据**之前将请求的数据导入缓存。
- 数据预取
  - 将数据加载到**寄存器**中 (HP PA-RISC loads)
  - 缓存预取：加载到**缓存**中（MIPS IV、PowerPC、SPARC V9)
  - **注意：**通常预取指令不允许导致故障；预取实际上也是属于推测执行的一种形式。
- 关于预取需要注意的地方：
  - 预取指令的开销 是否小于 减少缺失所带来的收益?
  - 更激进的超标量 可减轻 指令发射带宽的问题，但预取会造成更大的访存流量。

# 书上的例子

For the code below, determine which accesses are likely to cause data cache misses. Next, insert prefetch instructions to reduce misses. Finally, calculate the number of prefetch instructions executed and the misses avoided by prefetching. Let's assume we have an 8 KB direct-mapped data cache with 16-byte blocks, and it is a write-back cache that does write allocate. The elements of a and b are 8 bytes long since they are double-precision floating-point arrays. There are 3 rows and 100 columns for a and 101 rows and 3 columns for b. Let's also assume they are not in the cache at the start of the program.

```
for (i = 0; i < 3; i = i+1)
        for (j = 0; j < 100; j = j+1)
                a[i][j] = b[j][0] * b[j+1][0];
```

# 基于编译预取的优化

```
for (j = 0; j < 100; j = j+1) {
        prefetch(b[j+7][0]);
        /* b(j,0) for 7 iterations later */
        prefetch(a[0][j+7]);
        /* a(0,j) for 7 iterations later */
        a[0][j] = b[j][0] * b[j+1][0];};
for (i = 1; i < 3; i = i+1)
        for (j = 0; j < 100; j = j+1) {
                prefetch(a[i][j+7]);
                /* a(i,j) for +7 iterations */
                a[i][j] = b[j][0] * b[j+1][0];}
```

# GCC's support for data prefetch

In this post, we will explore together a very useful compiler built in function called __builtin_prefetch.

From the gcc official documentation

> This function is used to minimize cache-miss latency by moving data into a cache before it is accessed. You can insert calls to __builtin_prefetch into code for which you know addresses of data in memory that is likely to be accessed soon. If the target supports them, data prefetch instructions are generated. If the prefetch is done early enough before the access then the data will be in the cache by the time it is accessed.

In the previous post, we have shown two examples of memory access. One example was accessing the memory in a sequential manner, the other one was accessing the memory in random order.

We have shown that the sequential memory access was much faster due to the CPU cache and the Spatial Locality. This means the CPU was loading the next chunk of data in cache memory in advance.

in the random memory accessing example the CPU was loading the neighbouring data in cpu cache. but it was most of the time is loading the wrong data and getting cache misses.

Using this builtin prefetch function we can help the CPU by giving a hint on the expected next data to load.

Let's have the example from the pervious tutorial, and add a new function to it `SumRandomPrefFetch` which will use this built-in function.

https://ibrahimessam.com/posts/prefetch/

# Bonus: some papers

- Victim cache
- Skewed Associative Caches
- MLP-Aware Replacement
- Issues in Multicore Caches
- ……

# Victim Cache: 减少冲突失效

Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers

NP Jouppi - ACM SIGARCH Computer Architecture News, 1990 - dl.acm.org

Projections of computer technology forecast processors with peak performance of 1,000 MIPS in the relatively near future. These processors could easily lose half or more of their performance in the memory hierarchy if the hierarchy design is based on conventional ...

☆ 保存 99 引用 被引用次数：2180 相关文章 所有 31 个版本

- Norman Jouppi, "Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers," ISCA 1990.
- ISCA 25周年经典论文之一，建议课下阅读。

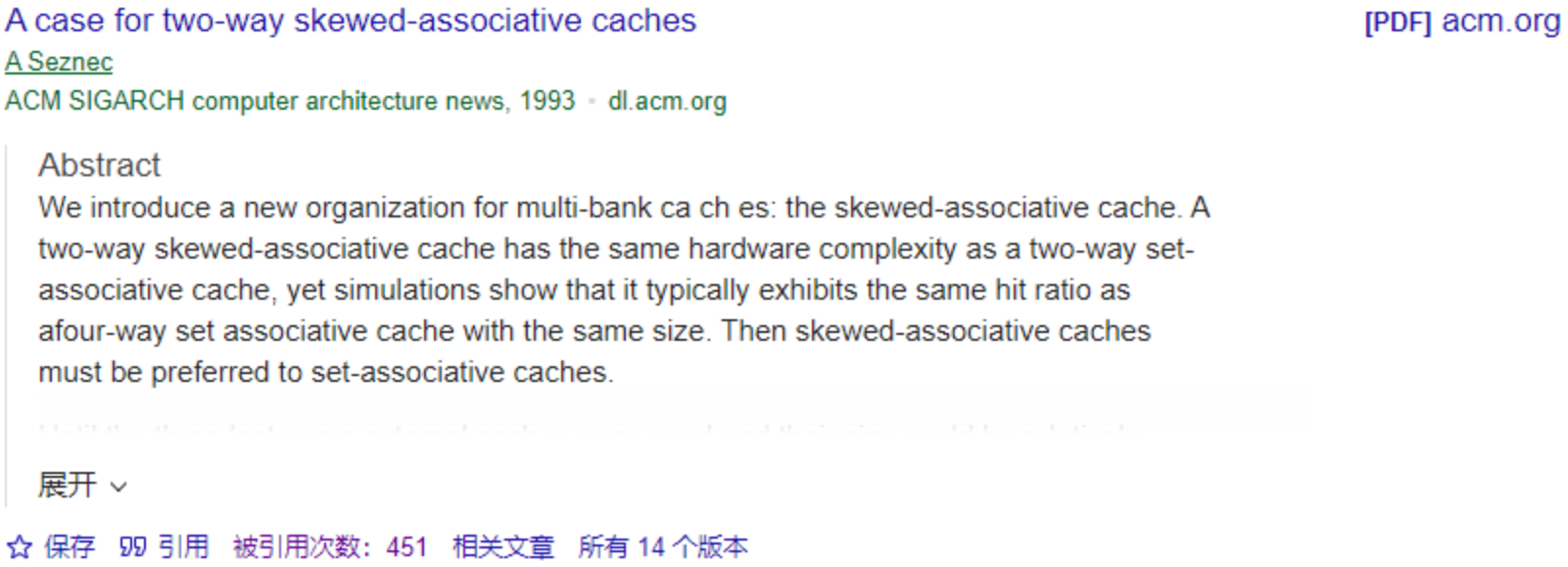
- **想法：**使用一个小的全相联的缓冲 (victim cache) 来保存被替换出去的block
  - \+ 可以消除映射到同一个Set里的多个block形成的乒乓效应
  - \- 如果与L2顺序访问，有可能增加缺失时的延迟;
  - \- 增加了设计的复杂度

# Hashing and Pseudo-Associativity

- Hashing: 使得index**更加离散**
  + 能够减少冲突缺失
    – Hashing使得blocks更加均匀地分布到sets里面
  - 哈希的计算复杂且耗时：可能会延长关键路径
- Pseudo-associativity (Poor Man's associative cache):
  – 将缓存组里的每个“way” 看作一个直接相联缓存
  – 每次只查找一个block，按顺序搜索各个way:
    - 当命中第k个way时，该way会被提升到第1的位置；原来的第1到k-1会被降低1个位置。
    - 当发生缺失时，新加入的block放到第1个way；第n个way被替换，原先第1到n-1个way往后移一个位置。

# Skewed Associative Caches

- **想法：**通过对**不同Way**使用不同的**索引函数**来减少缺失
- Andrew Seznec, "A Case for Two-Way Skewed-Associative Caches," ISCA 1993.


A case for two-way skewed-associative caches [PDF] acm.org

A Seznec

ACM SIGARCH computer architecture news, 1993 · dl.acm.org

Abstract

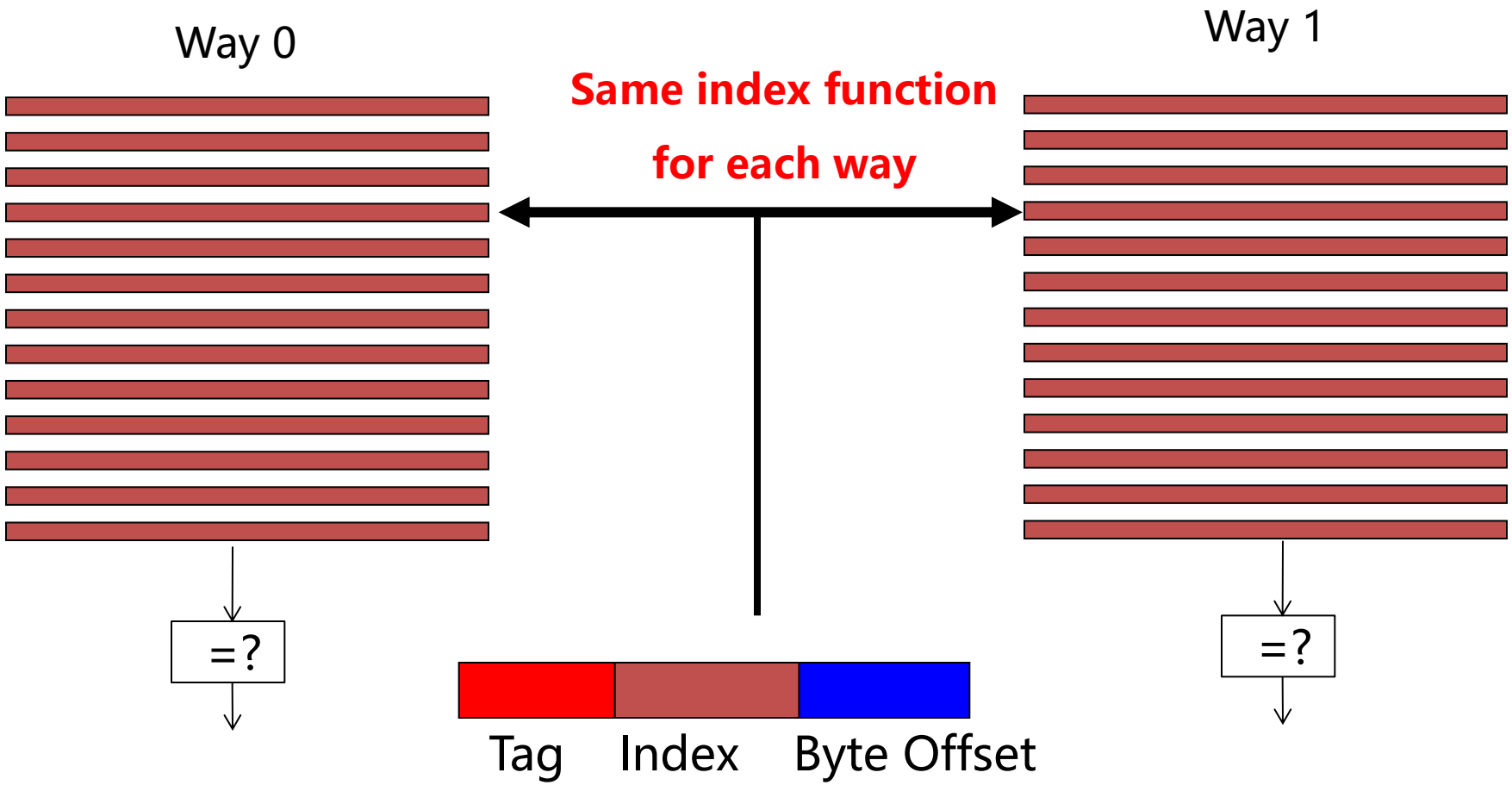
We introduce a new organization for multi-bank ca ch es: the skewed-associative cache. A two-way skewed-associative cache has the same hardware complexity as a two-way set-associative cache, yet simulations show that it typically exhibits the same hit ratio as afour-way set associative cache with the same size. Then skewed-associative caches must be preferred to set-associative caches.

展开

☆ 保存 99 引用 被引用次数：451 相关文章 所有 14 个版本

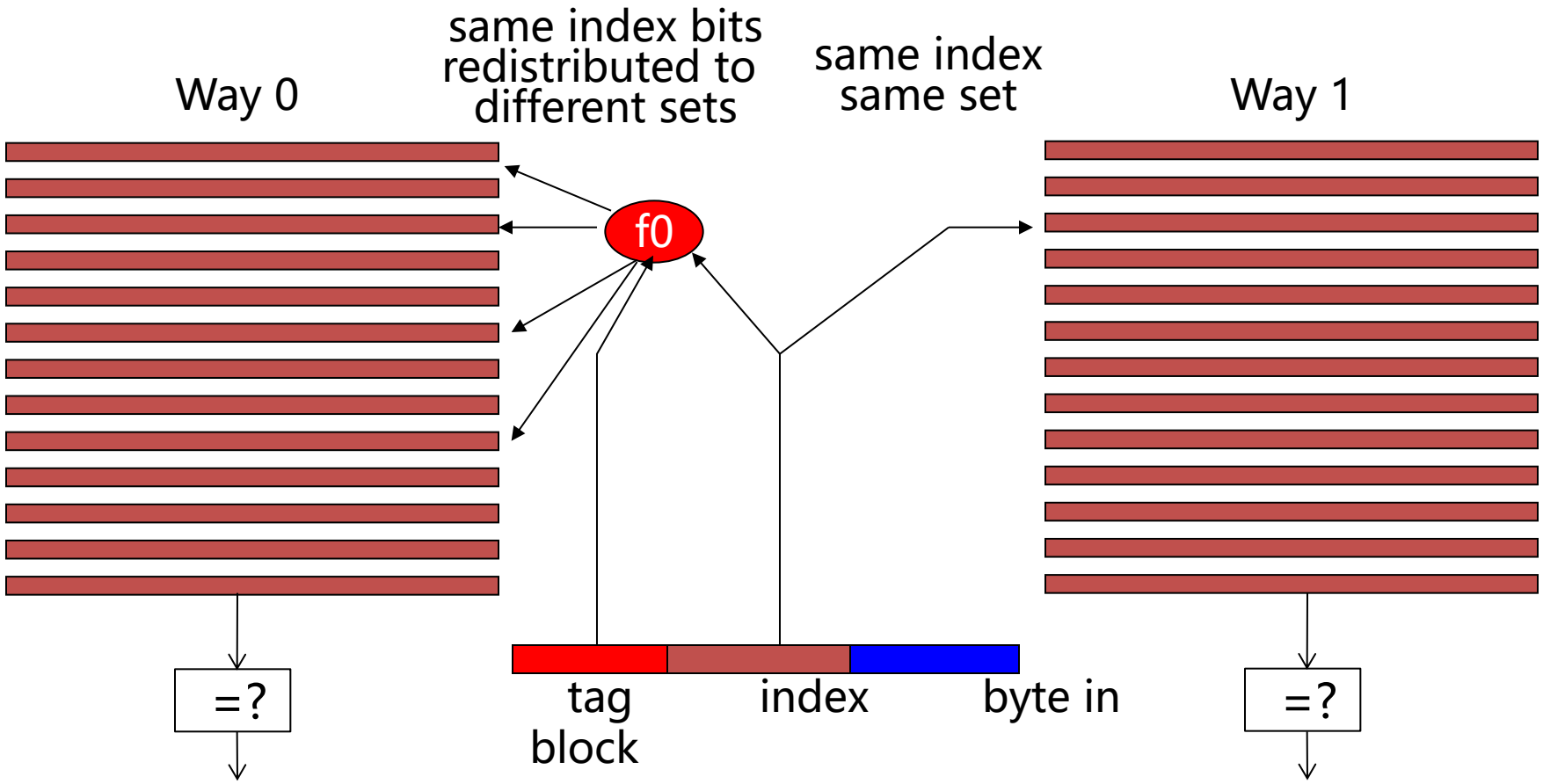
# Skewed Associative Caches (I)

- 基本的2-路组相联缓存的结构



# Skewed Associative Caches (II)

- Skewed associative caches
  - 使用不同的索引函数来访问各个Way



# Skewed Associative Caches (III)

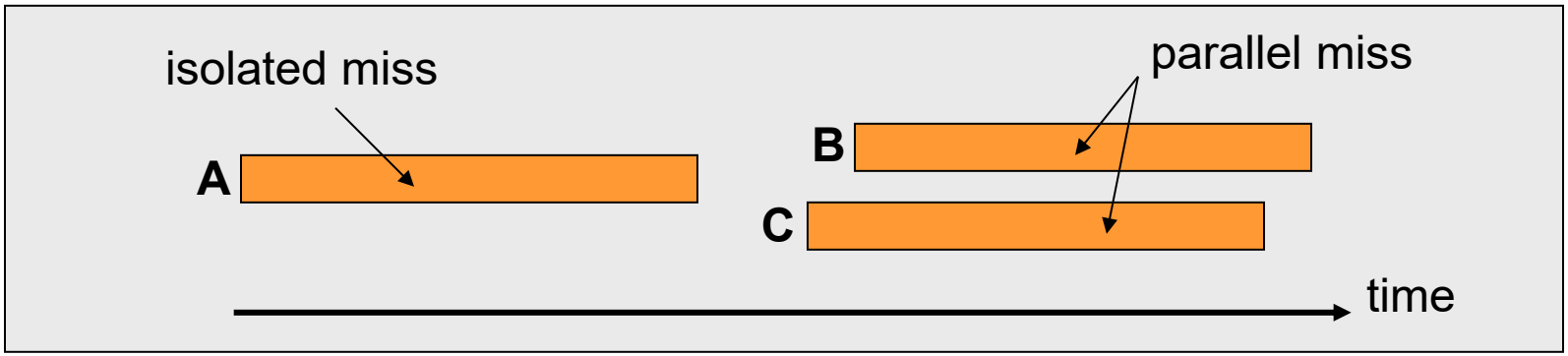
- **好处:** 索引经过哈希的处理之后，block的分布**更加离散** (内存的blocks能更加均匀地分布到缓存的Sets里) 。
  - 2个block被映射到同一个组（传统的组相联缓存中的组）的概率会大大降低
    - 减少了冲突缺失
- **开销：** 哈希函数的计算需要额外的延迟，以及较大的硬件开销。

# 缓存缺失的penalty分析

- 哪些因素会影响缺失的 延迟 (penalty)?
  - 缺失的block最终从哪里获取?
    - 本地 或者 远程的内存里?
    - 缓存处于存储层次的哪个层级
    - 行命中 还是 行缺失 （Row Buffer）
    - 内存控制器中排队? (Queue in Memory Controller)
    - …
  - 缺失造成处理器停顿的时间多长?
    - 该缺失能否与其它操作重叠处理?
    - CPU是否立刻就需要缺失的数据?
    - …

接下来，我们通过一个具体的示例，向大家展示当前系统中缓存性能的复杂性。

# 示例: 存储级并行性 (MLP)

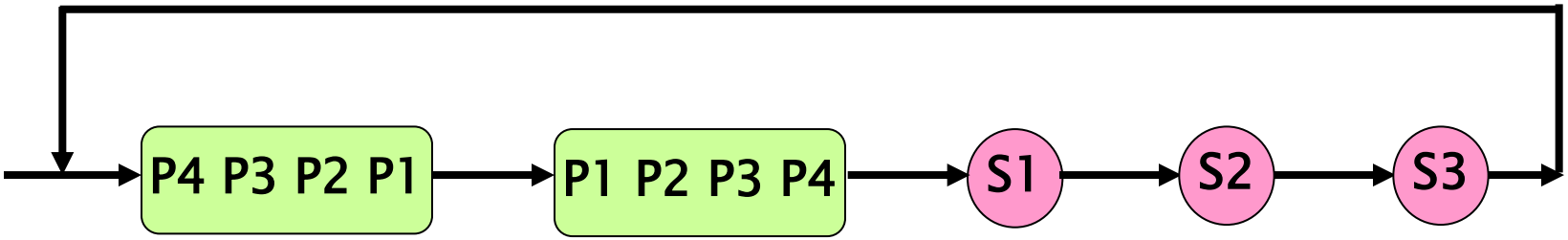



- Memory Level Parallelism (MLP) 指并行产生和服务多个内存访问 [Glew'98]
- 有若干个技术可以提升MLP
  - 例如：乱序执行、同时多线程
- 所以，**MLP是可变的**，有些缺失是独立的，而有些缺失可并行处理。
- 已有的缓存替换算法能感知以上特征吗?

# 传统的缓存替换算法

- 传统的策略均尝试减少缓存访问缺失的数量 (纯粹的缺失数量)
- **隐含的假设：**减少缺失的数量总能获得最好的存储访问时间的减少。[当前，不再总是正确。]
- 然而，实际情况下不同缺失的开销不同，MLP不同，从而使得上面所说的**隐含的假设**不成立!
  - 与可并行处理的缺失相比，减少一个独立的缺失对性能的帮助更大
  - 换句话说，减少一个长延迟的缺失比减少一个短延迟的缺失带来的收益更大

# 一个具体的DEMO



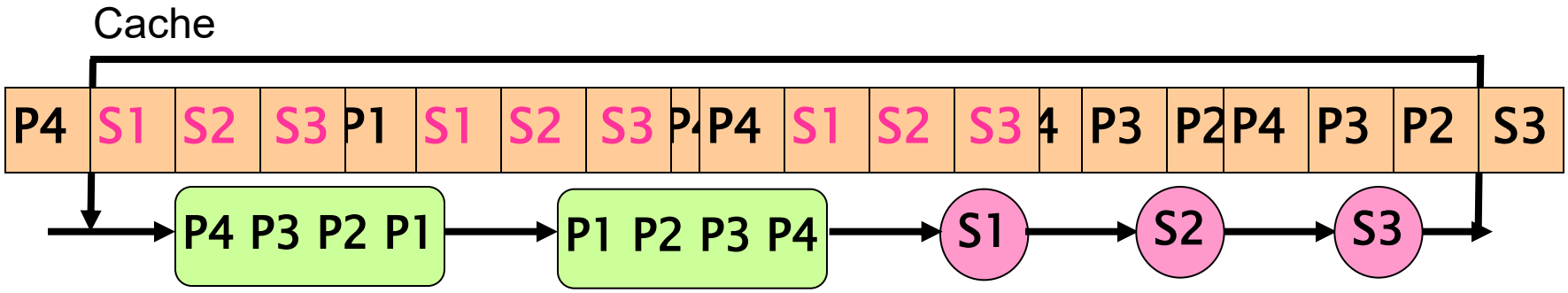

访问块： P1, P2, P3, P4 发生缺失可以并行处理

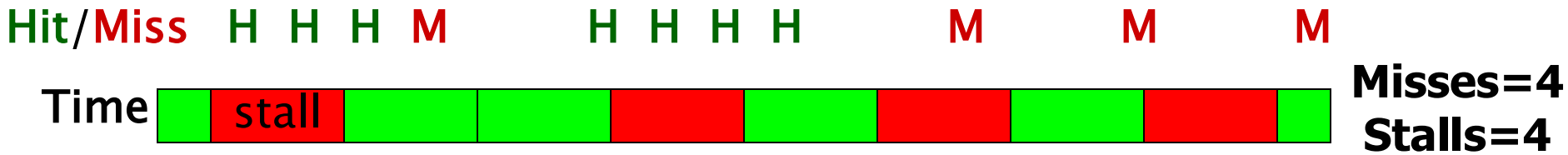
访问块： S1, S2, S3 发生缺失仅能独立处理

**思考：**基于一个具有4个block的**全相联缓存**，请分析下面2个替换算法在上述访问场景下的性能。

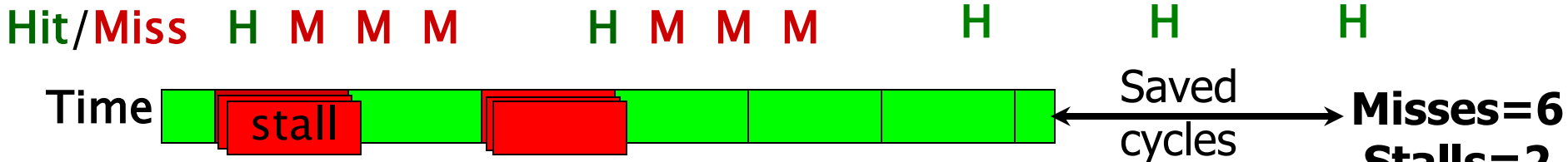
1. 最小化缺失数量 (Belady's OPT)
2. 减少独立缺失数量 (MLP-Aware)

# 最少的缺失数≠最好的性能




Belady's OPT replacement




MLP-Aware replacement

# MLP感知的替换策略

- 如何将MLP信息融合到替换算法的决策中?
- Qureshi et al., "A Case for MLP-Aware Cache Replacement," in ISCA 2006.
  - 建议课下阅读，作为系统方向的科研启蒙。
    - 发现问题
    - 解决问题
    - 写成论文
  - 这是更好的replacement策略的示例



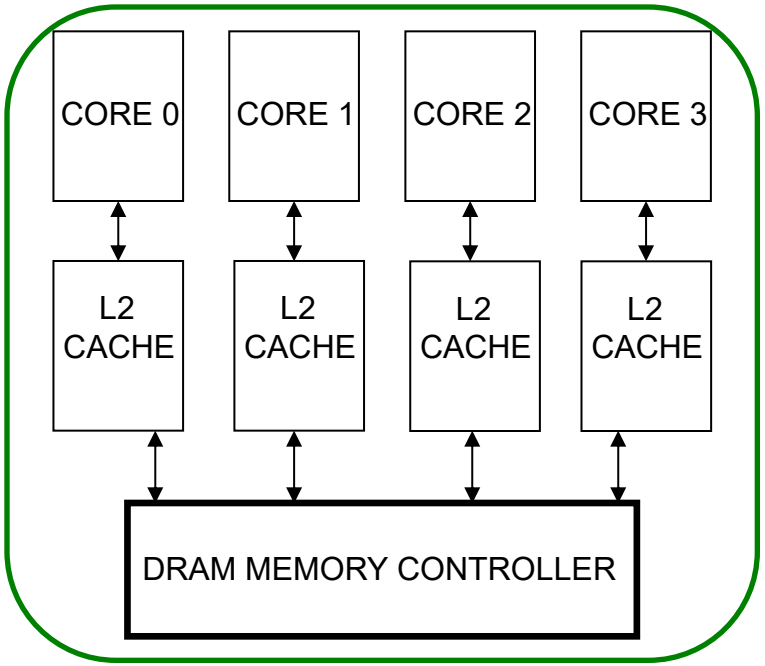
# 多核处理器中的缓存

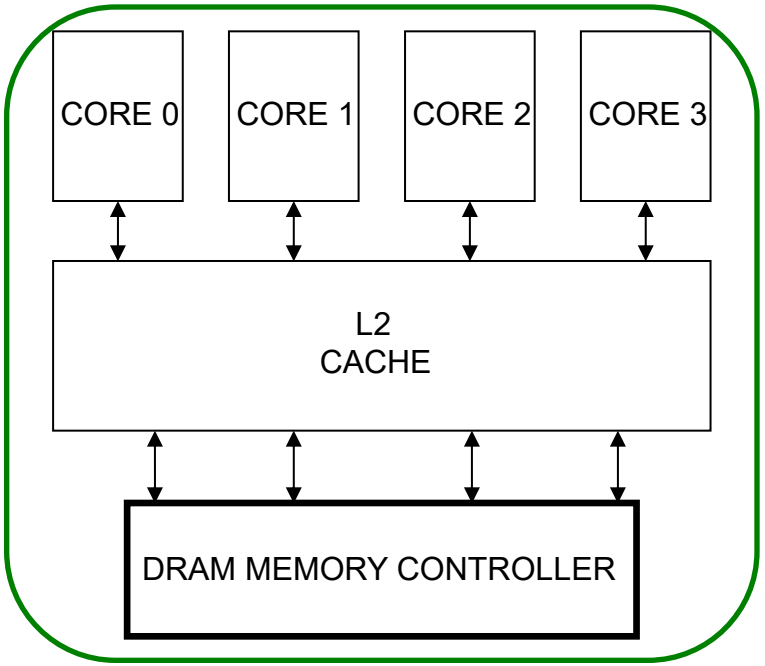
- 对于多核/多线程系统来说，缓存更加重要：
  - 内存带宽更加稀缺!!!
  - 缓存空间也更加有限!!!
- 多核处理器中缓存会面临哪些新的问题？
  - 共享 还是 私有？
  - 在共享缓存下，如何给线程分配空间？
  - 缓存管理算法需要考虑线程的特征吗？
  - 在共享缓存下，如何给不同线程提供QoS？
  - ……



# Private vs. Shared

- **私有缓存**：缓存仅属于唯一的核 (那么，一个共享的block可能在多个缓存中存在副本。=>会导致什么问题？)
- **共享缓存**：缓存空间由多个核共享（优缺点是什么？）




Q：你有写过多线程程序吗？

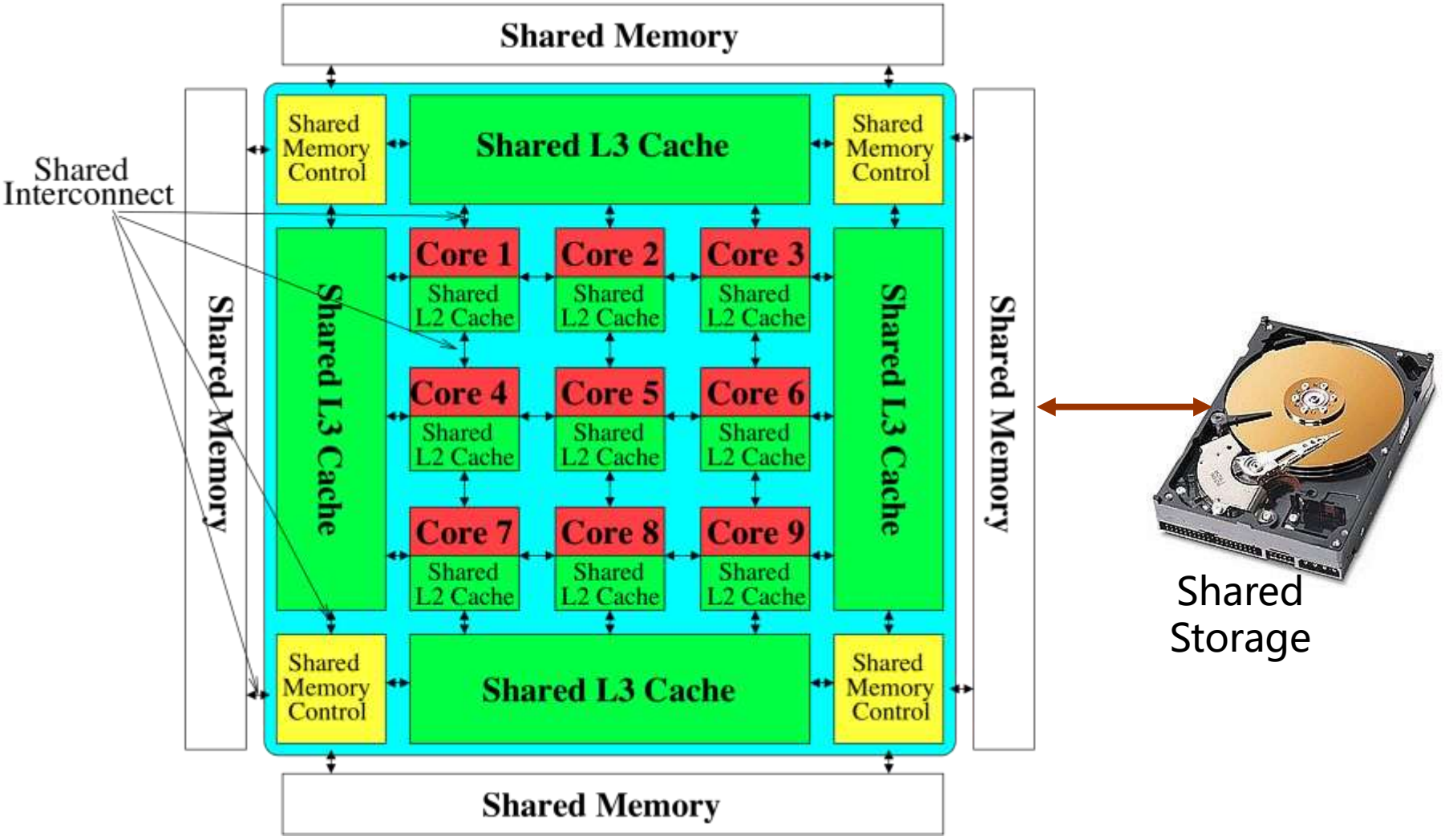
# 资源共享及其优点

- **想法：**让多个contexts共享硬件资源
  - 典型的硬件资源: functional units, pipeline, caches, buses, memory, etc.
- **资源共享的优点：**
  - \+ 可以提升资源的利用率和效率 → 更好的吞吐率
    - 一个线程不用某个资源时，另外一个线程可以使用；
    - 减少资源的重复配置 或者 过量配置 (如：随处可见的共享单车)
  - \+ 减少线程间的通信延迟
    - 例如，多个线程所访问的共享数据可以放在缓存中
  - \+ 与共享存储器模型 (Shared Memory Model) 兼容

# 资源共享的缺点

- 资源共享可能会造成资源冲突（如：共享单车）
  - 当一个共享的资源被占据时，其它线程无法使用该资源；
  - 没有一个合理的资源分配机制，可能会导致不公平。
- 所以，共享有可能会降低线程的性能
  - 与单独运行的场景相比，所获得的性能较差。
- 消除了性能隔离 → 程序多次运行所获得的性能不一致。
  - 线程性能与共同运行的线程 (co-runnner) 特性相关
- 没有控制的共享会损害QoS（服务质量）
  - 造成不公平，甚至饥饿；
  - 可能带来资源和能耗的浪费；
  - 这里所描述的问题，对当今的数据中心非常重要。

# 多核系统中的共享资源


Shared Memory
Shared Memory Control
Shared L3 Cache
Shared Memory Control
Shared Interconnect
Shared Memory
Shared L3 Cache
Core 1
Core 2
Core 3
Shared L2 Cache
Shared L2 Cache
Shared L2 Cache
Core 4
Core 5
Core 6
Shared L2 Cache
Shared L2 Cache
Shared L2 Cache
Core 7
Core 8
Core 9
Shared L2 Cache
Shared L2 Cache
Shared L2 Cache
Shared L3 Cache
Shared Memory
Shared Memory Control
Shared L3 Cache
Shared Memory Control
Shared Memory
Shared Storage

# 多核系统中的共享资源

- Memory subsystem (多线程和多核系统)
  - Non-private caches
  - Interconnects
  - Memory controllers, buses, banks
- I/O subsystem (多线程和多核系统)
  - I/O, DMA controllers
  - Ethernet controllers
- Processor (多线程系统)
  - Pipeline resources
  - L1 caches












# 多核系统需要QoS和共享资源管理

- 为何性能不可预测 (或者缺乏QoS) 是一个坏的设计？ (请思考!)
- 给程序员造成很多困难
  - 经过精心优化的程序可能获得较差的性能 (且其性能依赖co-runner的特征)
- 造成用户不满意 [datacenter场景]
  - 系统中的重要程序可能因迟迟获取不了资源而饥饿
  - 当共享的硬件资源不可控时，如何确保SLA (Service Level Agreement)？
  - 从技术层面来说，很难使用户满意。

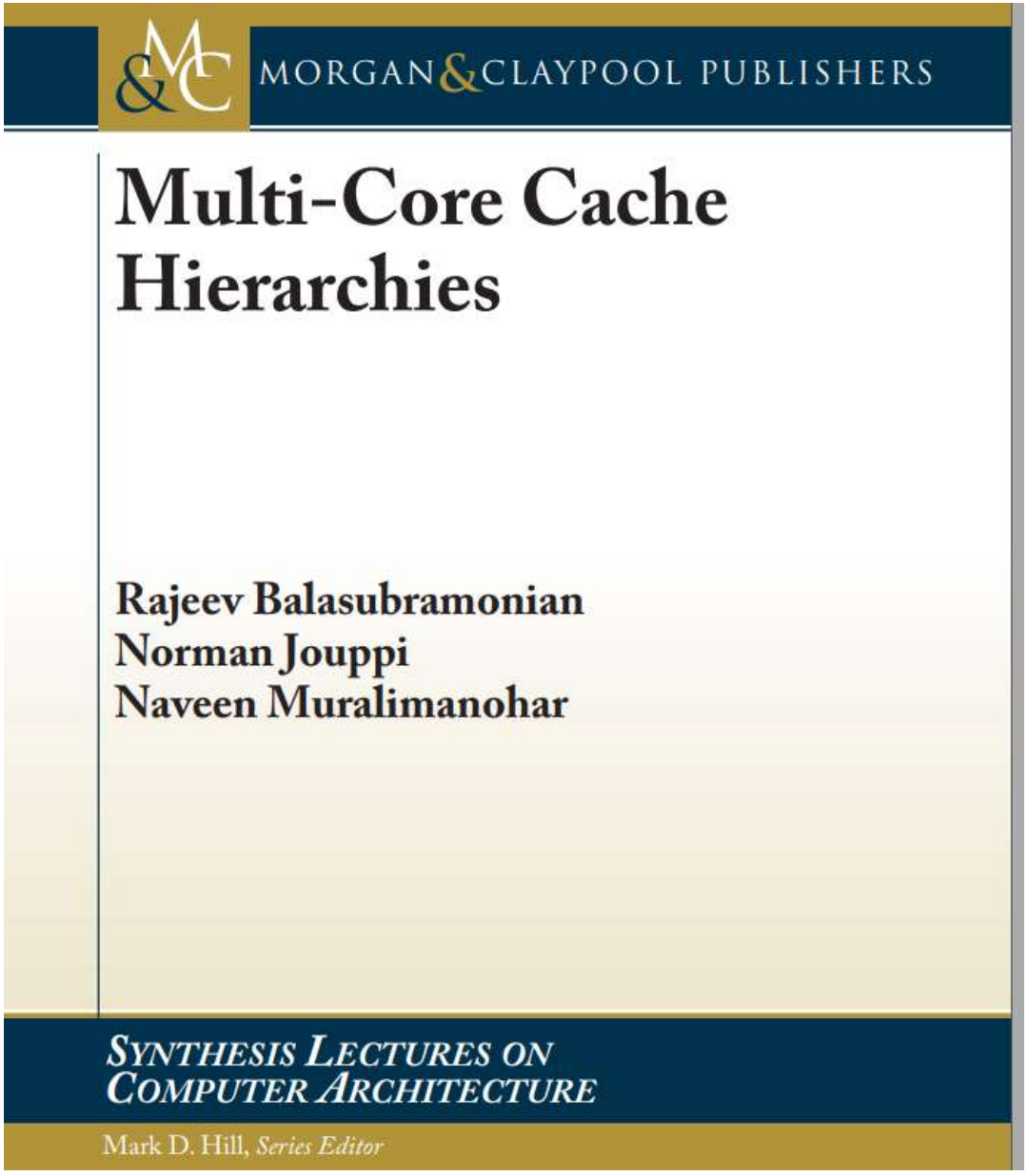
# Demo：共享缓存

- 优点：
  - 可以有效提升缓存空间的利用率
  - 可以对可用缓存空间进行动态划分
    - 没有静态空间划分的碎片问题
  - 容易维持一致性 (single copy)
  - 共享的数据在**缓存间**没有乒乓问题
- 缺点：
  - 访问速度慢 – 因为共享的缓存较大
  - 其它核的访问可能会造成冲突缺失
    - 核间冲突造成的缺失
    - 一些核可能会损害其它核访问缓存的命中率
  - 维持/保障每个核的服务质量 (或公平性) 很难
    - 分配多少缓存空间，多少带宽?
    - 如何提供隔离盒保护?

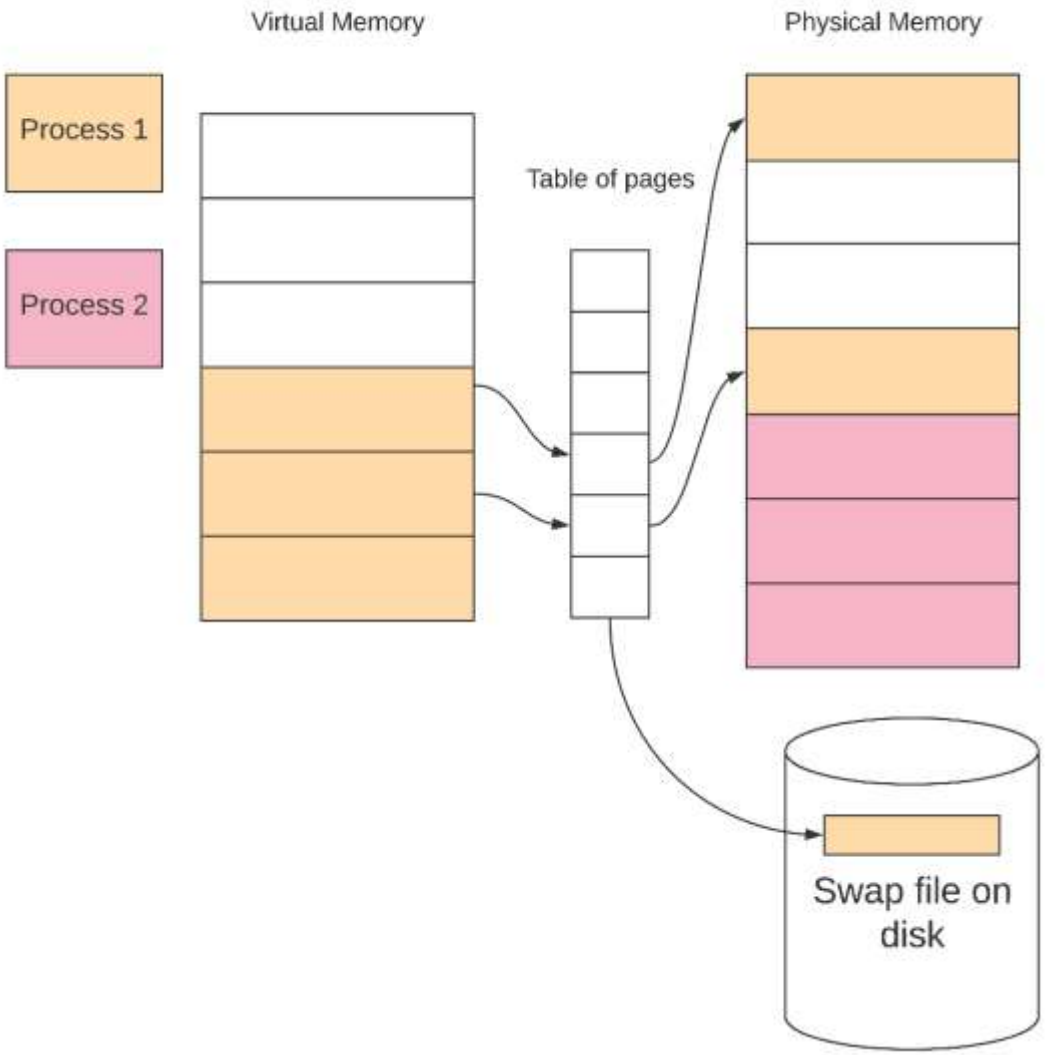
# Demo：共享缓存如何共享?

- 自由共享
  - 与单核系统采用一样的放置和替换策略 (通常是LRU 或者 pseudo-LRU)
  - 对thread/application的访存特征是盲目的
  - 一个新加入的block在替换victim时，对该victim属于哪个thread是盲目的。【盲目是不好的，不能提供QoS】
- 存在的问题【不能按需分配】
  - 缓存的利用率不高: LRU不是最好的策略
  - 缓存不友好的应用会破坏缓存友好型应用的性能
  - 不是所有的应用能从相同容量的缓存中获得相等的收益: 自由共享可能给获益较少的应用分配更多的缓存空间。

# 多核系统中的缓存架构


MORGAN & CLAYPOOL PUBLISHERS
Multi-Core Cache Hierarchies
Rajeev Balasubramonian
Norman Jouppi
Naveen Muralimanohar
SYNTHESIS LECTURES ON COMPUTER ARCHITECTURE
Mark D. Hill, Series Editor

# Next Topic


Virtual Memory
Physical Memory
Process 1
Process 2
Table of pages
Swap file on disk